

Azure Function Policy Orchestration

Direct Implementation Instructions

Use this sequence to build and test the first working version of the Policy Assistant orchestration service.

Step 1 - Create the Azure Function project

Open PowerShell and run:

```
mkdir azure-policy-assistant-function
cd azure-policy-assistant-function

py -m venv .venv
.venv\Scripts\Activate.ps1

func init --worker-runtime python --model V2
func new --template "HTTP Trigger" --name policy_chat
```

Step 2 - Install the required Python packages

Run:

```
pip install azure-functions
pip install azure-search-documents
pip install azure-identity
```

Make sure requirements.txt contains:

```
azure-functions
azure-search-documents
azure-identity
```

Step 3 - Add Azure AI Search settings

Open local.settings.json and add:

```
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "python",
    "AZURE_SEARCH_ENDPOINT": "https://YOUR-SEARCH-SERVICE.search.windows.net",
    "AZURE_SEARCH_INDEX_NAME": "YOUR-INDEX-NAME",
    "AZURE_SEARCH_CONTENT_FIELD": "YOUR-CHUNK-FIELD",
    "AZURE_SEARCH_VECTOR_FIELD": "YOUR-VECTOR-FIELD",
    "AZURE_SEARCH_SOURCE_FIELD": "YOUR-SOURCE-FIELD"
  }
}
```

```
}  
}
```

Replace every YOUR-... value with the actual values from your Azure AI Search service.

Step 4 - Log into Azure locally

Run:

```
az login
```

Use the Azure account that has access to your Azure AI Search service.

Step 5 - Verify Azure AI Search permissions

Confirm your user account can query the Search index.

For the deployed Azure Function, assign its managed identity:

```
Search Index Data Reader
```

Step 6 - Verify query-time vectorization

Open your Azure AI Search index and check:

```
Vector Search  
↓  
Profiles  
↓  
Vectorizer
```

If a vectorizer is configured, continue to Step 7. If there is no vectorizer, configure the Function to generate the query embedding before calling Search.

Step 7 - Replace function_app.py with starter code

```
import json  
import logging  
import os  
import uuid  
  
import azure.functions as func  
from azure.identity import DefaultAzureCredential  
from azure.search.documents import SearchClient  
from azure.search.documents.models import VectorizableTextQuery
```

```
app = func.FunctionApp()
```

```

SEARCH_ENDPOINT = os.environ["AZURE_SEARCH_ENDPOINT"]
SEARCH_INDEX_NAME = os.environ["AZURE_SEARCH_INDEX_NAME"]
CONTENT_FIELD = os.environ["AZURE_SEARCH_CONTENT_FIELD"]
VECTOR_FIELD = os.environ["AZURE_SEARCH_VECTOR_FIELD"]
SOURCE_FIELD = os.environ["AZURE_SEARCH_SOURCE_FIELD"]

```

```

credential = DefaultAzureCredential()

```

```

search_client = SearchClient(
    endpoint=SEARCH_ENDPOINT,
    index_name=SEARCH_INDEX_NAME,
    credential=credential
)

```

```

@app.route(
    route="policy-chat",
    methods=["POST"],
    auth_level=func.AuthLevel.FUNCTION
)

```

```

def policy_chat(req: func.HttpRequest) -> func.HttpResponse:

```

```

    try:
        body = req.get_json()

```

```

        message = body.get("message")
        conversation_id = body.get("conversationId")

```

```

        if not message:
            return func.HttpResponse(
                json.dumps({"error": "message is required"}),
                status_code=400,
                mimetype="application/json"
            )

```

```

        if not conversation_id:
            conversation_id = str(uuid.uuid4())

```

```

        vector_query = VectorizableTextQuery(
            text=message,
            k_nearest_neighbors=10,
            fields=VECTOR_FIELD
        )

```

```

        results = search_client.search(
            search_text=message,
            vector_queries=[vector_query],
            select=[CONTENT_FIELD, SOURCE_FIELD],
            top=5
        )

```

```

policy_results = []

for result in results:
    policy_results.append({
        "content": result.get(CONTENT_FIELD),
        "source": result.get(SOURCE_FIELD),
        "score": result.get("@search.score")
    })

return func.HttpResponse(
    json.dumps({
        "conversationId": conversation_id,
        "message": message,
        "results": policy_results
    }),
    status_code=200,
    mimetype="application/json"
)

except Exception as ex:
    logging.exception(ex)

return func.HttpResponse(
    json.dumps({"error": "Internal server error"}),
    status_code=500,
    mimetype="application/json"
)

```

Step 8 - Start the Azure Function locally

Run:

```
func start
```

Look for:

```
http://localhost:7071/api/policy-chat
```

Leave the Function running.

Step 9 - Send a test request

Send a POST request to:

```
http://localhost:7071/api/policy-chat
```

Use this JSON body:

```
{
  "conversationId": "test-001",
  "message": "What is the corporate remote work policy?"
}
```

Step 10 - Verify the Search response

You should receive a response similar to:

```
{
  "conversationId": "test-001",
  "message": "What is the corporate remote work policy?",
  "results": [
    {
      "content": "Employees may work remotely...",
      "source": "remote-work-policy.pdf",
      "score": 0.032
    }
  ]
}
```

Do not move to the LLM integration until this works.

Step 11 - Test several policy questions

- What is the corporate remote work policy?
- How frequently do employees need to change passwords?
- What is the incident escalation procedure?
- What policies apply to cloud costs?

Confirm that the correct policy chunks are returned.

Step 12 - Add the Foundry LLM package

After Search retrieval works, run:

```
pip install openai
```

Add this line to requirements.txt:

```
openai
```

Step 13 - Add the Foundry LLM settings

Add these values to local.settings.json:

```
"AZURE_OPENAI_ENDPOINT": "YOUR-ENDPOINT",  
"AZURE_OPENAI_DEPLOYMENT": "YOUR-LLM-DEPLOYMENT"
```

Use the deployment name of your Policy Assistant LLM.

Step 14 - Change the Function workflow

```
User Question  
↓  
Azure AI Search  
↓  
Retrieve Policy Chunks  
↓  
Build LLM Context  
↓  
Send Context + Question to Foundry LLM  
↓  
Receive Reasoned Answer  
↓  
Return Answer + Sources
```

Step 15 - Build the LLM context

```
context_parts = []  
  
for result in policy_results:  
    context_parts.append(  
        f""  
Source: {result["source"]}  
  
Policy Content:  
{result["content"]}  
""  
    )  
  
policy_context = "\n".join(context_parts)
```

Step 16 - Create the Policy Assistant system prompt

```
system_prompt = ""  
You are a Corporate Policy Assistant.  
  
Answer the user's question using only the supplied corporate  
policy context.  
  
Do not invent policy information.  
  
If the supplied policy documents do not contain enough
```

information to answer the question, state that clearly.

Identify the policy documents used to answer the question.
""

Step 17 - Send the context and question to the LLM

Send these three items to your Foundry LLM:

- System prompt
- Retrieved policy context
- User question

Step 18 - Change the Function response

Return the reasoned answer and sources instead of raw Search results:

```
{
  "conversationId": "test-001",
  "answer": "Employees may work remotely when...",
  "sources": [
    "remote-work-policy.pdf"
  ]
}
```

Step 19 - Connect the React chat client

Have the React application send:

```
{
  "conversationId": "abc-123",
  "message": "What is the remote work policy?"
}
```

to:

POST /api/policy-chat

Display answer in the assistant message bubble. Display sources underneath as citations.

Recommended Implementation Order

Complete these first

- ☐ Create Azure Function project
- ☐ Install Azure packages
- ☐ Configure Search settings

- ☐ Authenticate to Azure
- ☐ Verify query-time vectorizer
- ☐ Add Search starter code
- ☐ Run Function locally
- ☐ Successfully retrieve policy chunks
- ☐ Test multiple policy questions

Then complete

- ☐ Install OpenAI SDK
- ☐ Configure Foundry LLM
- ☐ Build policy context
- ☐ Send context to the LLM
- ☐ Return reasoned response
- ☐ Return policy citations
- ☐ Connect React chat UI

Do not add conversation storage, Cosmos DB, agents, a Microsoft Knowledge Base, or extra orchestration frameworks yet.

First get this working:

```

React
↓
Azure Function
↓
Azure AI Search
↓
Policy Results

```

Then add:

```

React
↓
Azure Function
↓
Azure AI Search
↓
Policy Context
↓
Foundry LLM
↓
Answer + Citations
↓
React

```